- Misclassification Error : Impurity
  GINI : Impurity
  Entropy : Impurity
  minimize

IG → Purity
IG Ratio → Purity
maximize

- Parts of building a classifier
  - → how to measure purity? (measure the best split)
  - → how to split data?
  - → when to stop splitting.
  - → how to measure → tie?

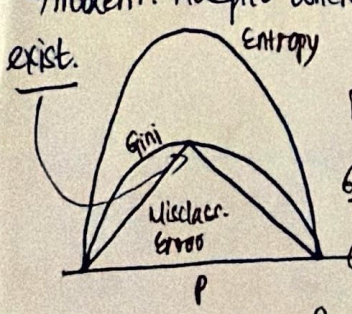
↑ subgroups, ↑ The groups naturally tend to become pure.

- we dont care about value of the function. We only want parameter which makes a function minimum (convex optimization)
- threshold = arg_min {avg. impurity}
- Misclassification Error (Node t)

$$Error(t) = 1 - \max_c P(c|t)$$

$C_A = 1$, $C_B = 5$
$P(C_A) = 1/6$, $P(C_B) = 5/6$
$1 - \max(1/6, 5/6) = 1 - 5/6 = 1/6$

maximum $(1 - \frac{1}{n_c})$ when records are equally distributed among all classes [implying least interesting info. M.C. Gini worst impurity (high badness)]
minimum (0.0) when all records belong to one class (implying most interesting info) [M.C. Gini → most purity (low badness)]
→ Problem: At split where prob = 0.5, derivative doesn't exist.



p = prob. of ClassA in that node.

Gini :→ $1 - \sum_j [P(j|t)]^2$

$C_A = 1$ $C_B = 5$ | $P(C_A) = 1/6$ $P(C_B) = 5/6$

$$Gini = \left(1 - \left((1/6)^2 + 5/6)^2\right)\right) = 1 - \frac{26}{36} = \frac{10}{36}$$

- Entropy : $-\sum_j P(j|t) \log_2 P(j|t)$
  → measures inhomogeneity of a node.
  → max^m : $\log n_c$ – records equally dist. among all classes – least info
  min^m : 0 – records – one class → most info.

Weighted entropy ~ GINI by sum of all absolute eigenvalues
$P(C_A) = 1/6$
$P(C_B) = 5/6$
Entropy :→ $-\frac{1}{6}\log_2\frac{1}{6} - \frac{5}{6}\log_2\frac{5}{6}$ → then compute their cumulative sum

Splitting based on Info. gain →

$$Gain_{split} = Ent.(p) - \sum_{i=1}^{k} \frac{n_i}{n_T} Entropy(i)$$

Parent Node, split, k partitions, no. of records in node i

Measures how many nodes creates new info.

[Entropy of parent Node – Weighted Entropy of child Nodes]
→ choose split that achieves most red^n – maximizes gain.

(–ve: Prefers splits → ↑↑ partitions (small but pure))
↳ Info. Gain Ratio = $\frac{Gain_{split}}{Split\ Info}$ → $-\sum_{i=1}^{k} \frac{n_i}{n} \log_2 \frac{n_i}{n}$

(Adjusts info. gain by entropy of partitioning. Higher entropy partitioning is penalized)

Split data into nodes → Parent Entropy (p) → Entropy for each node → Avg. entropy across all (A) nodes (Mixed Ent.) → P – A (Info. Gain) → Splitting Entropy → $\frac{P-A}{\text{Splitting Entropy}}$

Decision Trees → Actionable intelligence make NN's.

→ DL, Noise ↓, × overfitting, speed up learning, improve clustering stability

- Avg entropy (2 Nodes) ⇒ $\frac{N_1}{Total} \times H_1 + \frac{N_2}{Total} \times H_2$
- Worst Entropy = 1

PCA → ① subtract mean values ② find cov. Matrix of attributes we care ③ find Eigenvectors values of covariance ④ project data onto first few eigen vectors.

- Projecting data onto vector makes data simpler → form of data compression → ↓ Dimensionality Red^n
- ① translate data to new reference origin
  ② convert proj. vector to unit vector
  ③ compute dot product of data × unit vector.

Proj Amount = (> dimensions converted when transformed by m-m only scales by λ.) – Also losing info.
(Actual : EV are main directions of variance / structure in data)
[How can we automatically find the major axis of ellipse? → Use PCA × find Eigenvalues / vectors

- Mean subtraction : centers data at origin
- Rotation (PCA) → aligns patterns with axes.
- Decision trees can capture these natural boundaries with simple shallow splits
- Karhunen – Loève Transf. ss q^n

Eigenvectors – form an orthogonal set of basis vectors to describe data with

No. of Eigenvalues used

• KD Tree: Excell at NN searches. Instead of searching every pt in dataset, it quickly eliminates entire regions of space (only improves efficiency, not meaning of clustering).

Ex: Recommender System: By representing users as points in high dimensional space (each dim = behavior), a KD tree can quickly identify the most similar user components w/o exhaustive searching.

• Agglomerative Clustering: (never use is → creates artificial dist above) Ex: n clusters (n data pts) — at each step we identify 2 most similar clus & merge them. (KEY DECISION → Linkage Methods) This continues till all pts are in 1 cluster.

→ Dendrogram: - Tree-like structure that shows how clusters merge at each step. [Can cut this tree at any height to get diff^t no. of clusters — which gives flexibility in choosing granularity of clustering)

→ Single Linkage: Shortest dist. b/w 2 pts (one from each cluster) — sensitive to outliers (long chain like clusters)

→ Complete Linkage: Max^m dist — compact, spherical clusters of similar diameter

Agglomeration needs linkage but NOT K-Means

|| algo — K Means (parallel but outliers) Random forests (train trees independently) (computationally)

→ Average Linkage → Too slow/messy (avg of all pairwise distances b/w points in 2 clusters)

→ If centroid of cluster too sparse — use Medoid (real datapt closest to centroid.)

• KNN Classifier slows down — selection of features, distance metric, K's value

Design decisions: No. of features, Amount of data, High K

• Agglomeration (bus routes) — L2 Single linkage, Penalise → left hand turns

• Best Proj^n Vector → Vector b/w two class centers but adjusted for the skew in the 2 distributions

• the DT where decisions are same at level of tree can be implemented using lookup table

• Which of the following classifiers elicate to set parameters when training — histograms, PDF, DT (adaboost), SVM, Threshold Classifier

• Only way to know which principal components are best ones to use for a classifier → N fold Cross Validation.

LDA assumptions (supervised)

→ Data is Gaussian

→ Covariance of both classes — same

→ Data: linearly separable

→ Balanced classes.

$w = S_W^{-1}(m_1 - m_2)$ (within class scatter matrix)

(What direction should we project data onto best separate the classes)

Failure Modes
- XOR Problem
- classes → X Normally dist.
- X linearly separable
- Differentiating Power (X Mean, ✓ Spread)

Conditional Prob. — sensitive to outliers

$P(A|obs) = P(A|H_1)P(H_1|A) \ldots \forall P(A)$

Posterior $P(A|B) = \frac{P(A \text{ and } B)}{P(B)} = \frac{P(B|A)\,P(A)}{P(B)}$ (likelihood, prior, normalizing constant)

[Can't have 0 in table — add noise) (CM-elaboration)

2 cases to add random noise

eg $P(\text{Sick}|obs) = \frac{\text{Ctut 1}}{\text{sick}} \times \frac{\text{Ci 2}}{\text{sick}} \times \frac{\text{sick}}{\text{Total (sick + Nonsick)}}$ (when using fitted calculate prior prob - & then conditional prob)

Parzen density estimation

[Statistical independence: $P(A \times B) = P(A)P(B)$

(If A is subset of B, then $P(A|B) \times P(A)$

$P(A|B) = \frac{P(A)}{P(B)}$

ANN Problems
- enough correct data
- Training weights.

(NN's & Vanishing gradient problem)

• Joint Prob measures size of whole universe

Conditional Prob: " " within smaller "

• Joint Prob of all observations doesn't matter

• Cond. Prob. are conditionally independent.

Joint Prob → replaced → Product of individual Prob.

→ Seeing what you want to see – forcing linear when its actually curved.

- Forget made classifiers → Used association mining (Apriori Principle) to discover frequent purchase patterns, converted these patterns to features, trained predictive models and tuned decision thresholds.
- Ethical violations & – Inferring sensitive personal info w/o consent, violating privacy expectations & enabling targeting intrusive
- They valid internally? → Tested predictions against known outcomes using internal ground truth and evaluated performance using accuracy & ROC curves.
- Why? → Red card die? To encourage identifiable transactions enabling better tracking, stronger customer loyalty and increased repeat purchases despite small margin loss.
- Cash shoppers? → Linking behaviour across devices, cookies, emails, phone numbers, receipts, returns, in-store & online interaction patterns.

→ KD Tree: Data structure that organizes points in multidimensional space to enable fast NN searches by pruning large regions of the search space.

Problem it solves – Effecient NN search in MD space.

Works → recursively partitioning space and pruning large regions during search; fast? → avoids checking every datapt in ~~search~~ dataset.

DBScan – used for density based clustering + outlier detection

Works: forms clusters from core pts, expands via density reachability; labels noise.

When to use: arbitrary shaped clusters, need noise handling

CRISP-DM/Data Mining Game: Role of Domain experts → Create meaningful features & sanitycheck results.

Ethics: Problem framing + data collection + deployment

Business Case: At start & before deployment

Telemetry data: instrumented behaviour/usage logs collected automatically (clicks, events, timings etc)

Effective operator characteristics

ROC curves → Used for choosing thresholds (TPR vs FAR)

→ Graphical calculator: Pick operating pt on curve based on acceptable trade off

→ ~~High~~ low FAR? When humans respond; too many false alarms people ignore it.

→ high FAR → cheap/fast guess – which can be followed up with another classifier.

- Random Noise: to filter; M-estimation (NB classifier)
  Random Noise added to give a pseudo categorical data that falls on the same location. (Used for datavisualization [compensates for zeroes in Factor Tables for NB])
- Point of diminishing returns → K Means / Eigenvalue analysis / Clustering analysis
- Identify worthless attributes
  ↳ correlation analysis, mutual information
- Why toss all? Reduce curse.. splitting a dendogram / dicing
- Natural no. of clusters? → Dendogram agglomeration.
- Agglomeration → merge history + dendogram – reveals
- Attributes are more impt than algorithms.
  [KMeans – generally faster than agglomeration]
- Flip Direction (error becomes 45%)
- How many eigen vectors?
- Eigen vectors / values → Covariance Matrix of data
- Voronoi → Can be used to classify unknown pts – by assigning class of nearest region (1NN)
- N-cross validation: split into N folds, train on N-1, test on 1; repeat, avg performance. Used for estimating generalization & selecting hyperparameters/models.
- Grid search – used when function can be evaluated but not differentiated → Since we can't use gradients, use key parameter values, measure cost & pick the best.

→ Convex optimization: Minimizing a convex objective function over convex feasible set. – It is reliable – "no bad local minima traps".

→ Grid search: Evaluate the function on a grid of parameter combinations. → 57% (3 values) → 57%

eg 4 parameters

(a) → full grid search → 3^4

(b) → Axially ~~partial~~ aligned : all other parameters fixed, one parameter changed.

after these 12 evaluations, we pick best single ← (4x3) parameter tweak, update that parameter & repeat process from new pt.

- If gradients are available, GD is effecient, if not but evaluation is possible, use grid/axially-aligned search.
- Cost Function
  - Objective – Most impt part – minimize/maximize
  - Regularization – Not care much about it, helps break ties.
- × Accidents
- ↓ Time to reach destination.

Cost = #Accidents + (1/10) * (Minutes to go to work)

(Use Eigen Vector – for classification × – PCA maximizes variance not class separation so it can mix classes)

- 1BNN vs 1NN: less sensitive to noise/outliers, smooth decision boundary.
- K in KNN → N cross validation + try values.
- Mediod: Actual datapt minimizing total distance.
- RF: many trees vote, ↓ Error.
- Prototype: representative point for a cluster (often centroid / mean)

• Parzen density estimation is non-parametric method to estimate chance / prob. of data being on one dimension. Used to estimate a PDF from data. To estimate density at query pt x, we
- look how close x is to each data pt.
- add up influence of nearby pts (large dataset)
- Normalize result.

[Used it for computing Naive Bayes for continuous variables]

→ While Naive Bayes is parametric, dist. assu: Gaussian, feature independence → reqd, small data, very fast computation

• Why one rule, one NN, Naive Bayes? - as most of the things are done using ANNs: Good common basis for comparison. Also, it takes very long to train ANNs for small achieving improvements in accuracy (ANNs → 87% acc → 3 weeks; decision Trees → 5 min → 85%)

• In many real world problems, no. of variables and their interactions are too large for hand-crafted rules. ML allows us to automatically learn these relationships from data.

• Why sigmoid? Easy to find its derivative

• ANN's problems: (I) Getting data & then labelling it. (II) Getting all the weights classified.

• Why Naive Bayes? Works well with high dimensional data. Conditional Independence assumption reduces complexity. It performs surprisingly well in text classification, spam detection, document categorization (Very easy to implement, decisions are mathematically transparent)

• Decision boundary moves towards other class - if prob of one distribution goes up.

• If variance of one distribution goes up, the decision boundary moves towards that class because higher variance spreads the class out reducing its peak likelihood near the centre.

Only way to know which Principal Components are best ones to use for classifier (try each projection & test resulting classifier using N-fold Cross Validation. (Polynomial - Decision boundary)

→ count no. of errors for all groups. (No. of groups = No. of LDA) (try & test all principal components)

• Used something like random forest - in assignment with fast rejection cascade - majority class selection at a single node inside one decision tree / cascade, not voting across multiple trees.

• XOR Problem: classification task where classes are not linearly separable. Each class occupies opposite corners of a square, so no single straight line can separate them.

• Logical Regression → We are not setting up a system of equations. Instead it is an iterative function.

→ Not any of these.

→ We take a function curve, shift it left/right until it does what we want it to do. Inside it, we use sigmoid function(s) = α (input) + shift. When we add something to function - it shifts whole thing to left. If we multiply by left, it makes it go up or down - faster or slower. $(s = f(\alpha x + \delta))$

[Sigmoid family: By combining various sigmoid function - we can get any function]

→ LR modifies α & δ until function gets closest to the desired curve.

• ANN ~~will~~ ~~works well~~ - when classes are well separated (However, it doesn't tell us anything about data we don't have) [ANNs have no advantage and may overfit compared to simpler classifiers]

• Fisher's Linear Discriminant doesn't iterate at all - it has a closed form (one-shot) solution.

• PCA finds direction of max^m variance w/o labels while FLD finds direction that best separates classes using labels.

• NBayes better than one-rule:-
↳ It considers more features, combines features, pre-observes the situation. And if data is elliptical and well separated, the prob. donot overlap.

• Pretty sparse - might want to use Medoid.

• Best way to decide no. of nodes in decision tree:-

• Project data onto one dimension & then use threshold.

• Voronoi polygons (Pizza) → 1NN

• Curse of dimensionality: If we go out 1SD (Mahal. dist) from centre of cluster, we collect only 20% of data

• Info. Gain Ratio compensates for natural tendency for classifiers to ~~feed~~ split on more categorical options. for ex → 11 shoesizes / 5 shirt sizes → Thus, splitting on 11 shoe sizes, there are 11 groups which are naturally more accurate than 5 shirt sizes. The info gain ratio compensates for this.

• Using all eigen vectors just rotates the data. (Thus, PCA is only useful when we discard low-variance eigen vectors) [FLD can improve classification even in 1D]

• Halo Effect: cognitive bias where an overall +ve/-ve impression of a person/object influences judgements about its specific traits when those traits are unrelated